HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY

CAPSTONE PROJECT REPORT:

Laptop shop Management Web Application

***Supervised by*:**

Ph.D. Bui Thi Mai Anh

***Presented by*:**

Au Trung Phong - 20225455

Nguyen Nam Khanh - 20225448

Vu Duc Minh - 20225514

Tran Sy Minh Quan - 20225521

Dao Vu Tien Dat - 20225479

Hanoi - Vietnam

2025

Contents

1 Introduction

1.1 Purpose

The purpose of our project is to provide an **efficient**, **user-friendly**, and **centralized platform** for managing a laptop retail business. This application addresses common challenges by offering digital tools to streamline operations. The main goals include:

- **Streamlining inventory management** – Track and update laptop stock easily, including model details, quantities, and specifications.
- **Simplifying sales processing** – Record transactions quickly and generate accurate sales reports to monitor business performance.
- **Improving customer service** – Maintain customer information and purchase history to enable personalized and effective communication.
- **Reducing human error** – Automate and digitize manual processes to increase reliability.

1.2 Scope

The project involves the development of a **full-stack web application** using modern technologies. The scope includes both front-end and back-end development, with features designed to meet the practical needs of laptop retailers. Key aspects include:

- **Front-end:** Built with **React** to provide a dynamic and responsive user interface.
- **Back-end:** Developed using **Python** to handle business logic, data storage, and system operations.
- **Core functionalities:**
  - **Inventory management**
  - **Sales and order processing**
  - **Customer relationship management**
  - **Administrative control panel**
- **Scalability:** Designed for future growth, including feature expansion and performance under increased usage.
- **Usability:** Aimed at users with varying levels of technical expertise.

1.3 Overview

This report provides a structured analysis of the project, detailing both its conceptual foundation and technical execution. It is organized into several sections that cover the following:

- **Project Objectives** – Describes the main goals the application aims to achieve in improving retail operations.
- **System Features** – Explains the core functionalities such as inventory management, sales processing, and customer tracking.
- **Use Cases** – Outlines typical user interactions with the system, illustrating how different features support real-world retail scenarios.
- **Technical Implementation** – Details the technologies used, system architecture, and development approach for both front-end and back-end.
- **Conclusion and Future Work** – Summarizes the project outcomes and suggests potential improvements or extensions.

Together, these sections offer a comprehensive view of the project's purpose, design, development process, and practical impact on laptop retail business management.

2 Key Features

The ITSE Laptop Shop Management Web Application provides a comprehensive toolset for both customers and administrators to manage all aspects of a laptop retail business. Built with a React front end and a Python back end, the system relies on a database to store and process product listings, orders, customer reviews, and refund requests. The following subsections outline each key feature and include concise descriptions of their purpose and user interactions.

2.1 Product Exploration and Search

This feature enables customers to efficiently browse and locate laptops using intuitive search and filter controls. The workflow is outlined below:

1. **Accessing the Catalog:** Customers open the customer portal and navigate to the laptop catalog page.

2. **Performing a Keyword Search:** They can enter any keyword (for example, “Dell” or “gaming”) to instantly narrow down the list to matching models.

3. **Applying Filters:** After—or instead of—typing a keyword, customers can apply filters such as brand, price range, screen size, or other criteria to further refine their results (e.g., “laptops under $1000” or “14-inch business laptops”).

4. **Viewing Results**

    - Once a search term is entered or filters are selected, the system immediately updates the displayed catalog to show only those laptops that meet the criteria.
    - Each visible item includes essential details—model name, price, availability status, and a thumbnail image—so customers can quickly compare options.

Summary of Benefits: By combining keyword search with dynamic filtering, this feature ensures that customers can locate their preferred laptops in just a few clicks, with results updating in real time as they adjust their search parameters.

2.2 Cart Management and Order Placement

This feature allows customers to manage their shopping cart and complete purchases with multiple payment options. The workflow is outlined below:

1. **Adding Items to Cart:** Customers add laptops to their cart from the product detail page.

2. **Reviewing and Modifying Cart:** The cart allows viewing, adjusting quantities, or removing items before checkout.

3. **Checkout and Payment Selection:** Customers place orders, choosing “Pay on Delivery” or “Pay via E-banking” through an external payment gateway.

4. **Order Confirmation:** After selecting a payment method and confirming, customers receive an order confirmation.

Summary of Benefits: This feature streamlines the purchasing process, ensuring customers can easily manage their selections and complete transactions with minimal effort.

2.3 Customer Reviews and Ratings

This feature allows customers to share feedback on purchased laptops, helping others make informed decisions. The workflow is outlined below:

1. **Submitting Reviews and Ratings:** Customers submit reviews and ratings for laptops they've bought, accessed from the product detail page.
2. **Entering Review Content:** Customers enter review text (e.g., "Excellent display") and select a rating (e.g., 5 stars), which are displayed publicly.
3. **Viewing Reviews:** Other customers can view reviews on the product page to assess product quality.

Summary of Benefits: The review feature encourages customer engagement and builds trust by showcasing authentic feedback.

2.4 Refund Ticket System

This feature allows customers to submit refund requests and administrators to process them efficiently. The workflow is outlined below:

1. **Submitting a Refund Request:**
    - Customers request refunds for orders through their account in the customer portal.
    - They specify a reason for the refund when submitting the request.
2. **Tracking Refund Tickets:** The system generates and tracks refund tickets, capturing order details and the current status of each request.
3. **Admin Management of Refund Tickets:**
    - Administrators view and manage all refund tickets from the admin panel.
    - Admins update the status of each ticket (e.g., "Approved" or "Rejected") after reviewing the request.

Summary of Benefits: This feature ensures efficient handling of returns and maintains customer satisfaction through clear, timely communication.

2.5 Inventory Management (Admin)

This feature enables administrators to keep the laptop inventory up-to-date using management tools. The workflow is outlined below:

1. **Adding New Laptops:**Admins add new laptops to the inventory by entering details such as model, brand, price, and stock quantity.
2. **Editing or Removing Existing Models:** The system allows admins to edit details of existing laptops or remove discontinued models from the catalog.
3. **Real-Time Visibility:** Any changes made to the inventory are immediately visible to customers in the catalog.

Summary of Benefits: Inventory management helps administrators maintain accurate product listings, supporting smooth retail operations and preventing stock discrepancies.

2.6 Dashboard and Order Status Management (Admin)

This feature provides administrators with insights and tools to manage order progress effectively. The workflow is outlined below:

1. **Dashboard Metrics:** Admins view a dashboard displaying key metrics such as total sales, pending orders, and low-stock items.
2. **Order Listing and Status Updates:** The system lists all orders, allowing admins to update order statuses (e.g., from “Pending” to “Processing”).

Summary of Benefits: This feature empowers administrators to monitor business performance through real-time metrics and keeps customers informed about their order progress, enhancing transparency and customer satisfaction.

2.7 User Authentication

This feature secures access for customers and administrators through registration and login. The workflow is outlined below:

1. **Customer Registration:** Customers register with an email and password to create an account.

2. **Login for Customers and Admins:** Both customers and administrators log in to access their respective portals (customer portal or admin panel).

3. **Admin Panel Access Control:** The system restricts admin panel access to authorized administrators only.

4. **Logout and Session Termination:** Users can log out to end their session, ensuring account security.

Summary of Benefits: Authentication ensures that features are accessible only to authorized users, with a clear separation between customer and administrator roles.

3 Use case

3.1 Overall Use Case

Figure 1: Overall Use Case of Laptop Shop Management System

3.2 5 Selected Use Cases

3.2.1 Use Case Diagram of 5 Selected Use Case

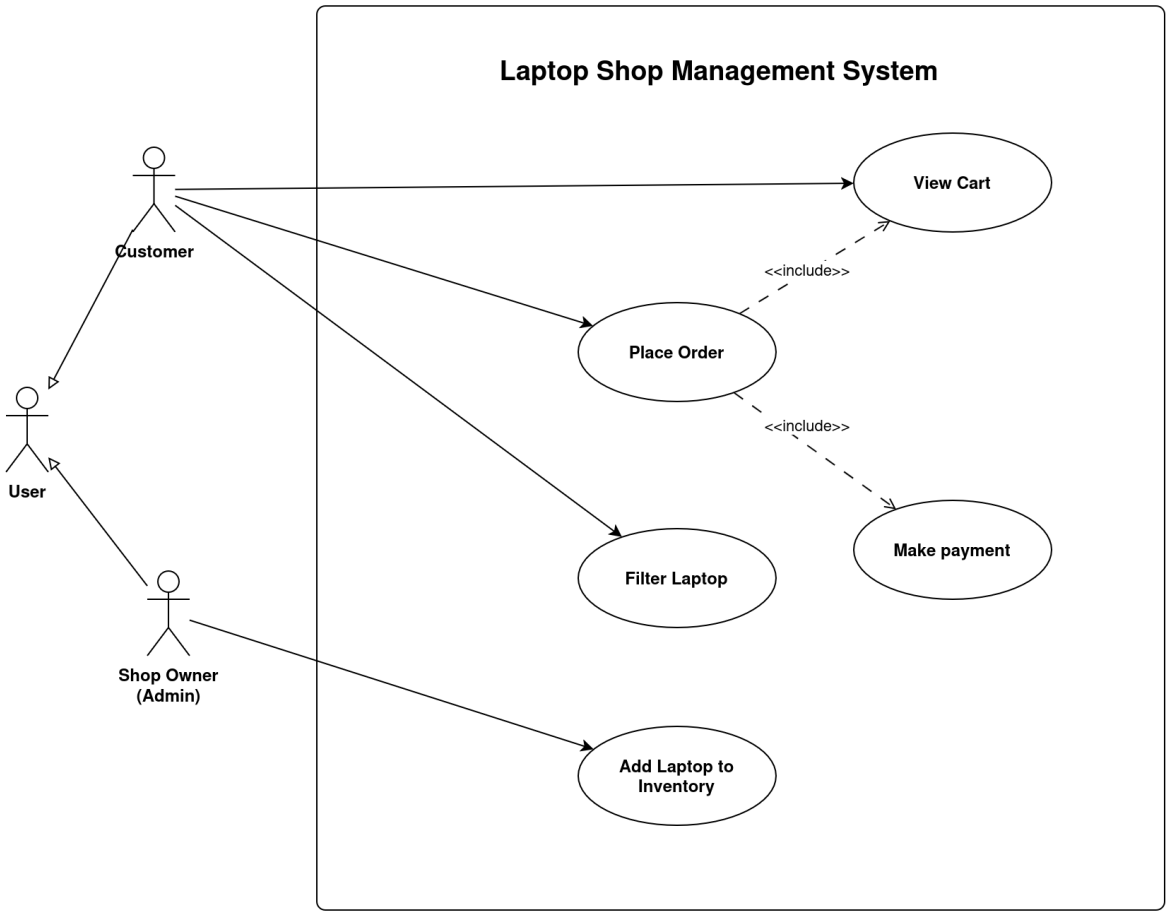

Figure 2: Use Case Diagram of 5 Selected Use Case

3.2.2 Details of 5 Selected Use Cases

Use Case ID	4	**Use Case name**	Filter Laptop
Description	Allows the customer to apply filters to narrow down the search results for laptops		
Actor	Customer		
Trigger	The customer initiates the action to filter laptops by selecting filter options		
Pre-Condition	None		
Post-Condition	• The customer successfully applies filters and finds laptops that match the selected criteria		
Basic Flow	Step	Performed by	Action
	1	Customer	Clicks on the *New Products* tab name on the homepage
	2	System	Navigates to the Catalog page displaying all products
	3	Customer	Selects the desired filter options (e.g., brand, price range)
	4	Customer	Clicks "Apply Filters" to update the search results.
	5	System	Displays the products that match the selected filters
Exception Flow	5a	System	If no matching results are found: displays no products
	5b	System	If a product is out of stock: do not display it in the filtered results
	5c	System	If there is a system error during product filtering (e.g., database failure), display an error message: "Unable to filter products. Please try again later."

Figure 3: Filter Laptop Use Case

Use Case ID	7	**Use Case name**	View Cart
Description	The customer views their shopping cart		
Actor	Customer		
Trigger	The customer initiates the action to view their cart		
Pre-Condition	• The customer must be logged in		
Post-Condition	• The customer is able to view the products currently in their cart		
Basic Flow	Step	Performed by	Action
	1	Customer	Clicks the *Cart* button on the page header
	2	System	Navigates to the cart page
	3	System	Displays all products in the customer's cart
Alternative Flow	1a	Customer	Clicks on the user icon
	1a1	System	Displays the user function menu
	1a2	Customer	Selects the "My Cart" option
Exception Flow	3a	System	If the cart fails to load: no products are displayed

Figure 4: View Cart Use Case

Use Case ID	9	**Use Case name**	Place Order
Includes	• `<<include>> View Cart` • `<<include>> Make Payment`		
Description	The customer reviews the cart, confirms shipping details, and completes payment. The scenario **always calls** the reusable logic of **View Cart** and **Make Payment**.		
Actor	Customer		
Trigger	The customer clicks **"Place Order"** on the *View Cart* page.		
Pre-Condition	• Customer is logged in • Cart is not empty		
Post-Condition	• Order is stored with **Pending** status • Stock is reversed/updated		
Basic Flow	Step	Performed by	Action
	1	`<<include>>` **View Cart**	Invokes the **View Cart** use case so the customer can review items.
	2	Customer	Clicks **Place Order**.
	3	System	Checks inventory, calculates total (items + shipping + discount).
	4	System	Show the details of the order and the Payment Methods
	5	Customer	Check and confirm the detail of the order
	6	`<<include>>` **Make Payment**	Invokes the **Make Payment use case** so the customer can finish the order
	7	System	Shows confirmation page.
Alternative Flow	5a	Customer	Notices incorrect order information, clicks on edit
	5a1	System	Places the order and displays the order confirmation page *End Use case*
Exception Flow	3a	System	If any item is out of stock → show error, return to **View Cart** for update.
	7a	System	Database failure while creating order → display "Could not place order, please try again."

Figure 5: Place Order Use Case

Use Case ID	10	**Use Case name**	Make Payment
Description	Handle all steps required to record payment for an order, delegating to the chosen payment method (Pay on Delivery or Pay via E-Banking) and updating the order status accordingly.		
Actor	Customer		
Trigger	This use case is invoked after Customer check his/her order's information		
Pre-Condition	• Customer checked the order's information		
Post-Condition	• The payment method is selected and done by customer		
Basic Flow	Step	Performed by	Action
	1	System	Shows payment options
	2	Customer	Selects a method
	3	System	Calls the selected specialised use case
	4	Specialised flow	Returns result
	5	System	Show confirmation
Exception Flow	2a	Customer	Aborts payment
	2a1	System	Show ***Payment Cancel*** notification *Use case ends here*
	5a	System	Error occurs during step 4, show ***Payment Failed*** notification

Figure 6: Make Payment Use Case

Use Case ID	17	**Use Case name**	Add Laptop to Inventory
Description	Create a new product entry with specifications, price, and images.		
Actor	Admin		
Trigger	The admin initiates the action to add a laptop to the inventory.		
Pre-Condition	• Admin is logged in with appropriate privileges.		
Post-Condition	• New laptop entry is saved and appears in the inventory list.		
Basic Flow	Step	Performed by	Action
	1	Admin	Clicks on the User icon
	2	System	Displays Menu
	3	Admin	Clicks on Add products
	4	System	Displays form for new laptop details
	5	Admin	Enters laptop specs, price, and uploads images
	6	Admin	Clicks Submit button
	7	System	Saves the new laptop to the inventory and confirms success.
Exception Flow	7a	System	Admin enters one or more fields in the wrong format: Highlight fields and show: **"Invalid input format."** *Use case back to Step 5*
	7b	System	If there is a system error during adding the laptop to the inventory (e.g., database failure), display an error message: "Unable to add laptop to inventory. Please try again later."

Figure 7: Add Laptop to Inventory Use Case

4 Technical Implementation

The ITSE Laptop Shop Management Web Application is built with a modern technology stack to deliver a seamless experience for managing laptop sales and operations. This section outlines the technical setup, starting with the Docker-based development environment, followed by the front-end, back-end, and database components. The implementation supports key features such as product exploration, order placement, customer reviews, refund tracking, inventory management, admin dashboard insights, and secure user access.

4.1 Frontend

4.1.1 Component Architecture

This section outlines the frontend components architecture, divided into **Single Pages**, **Grouped Pages**, and their **Components**.

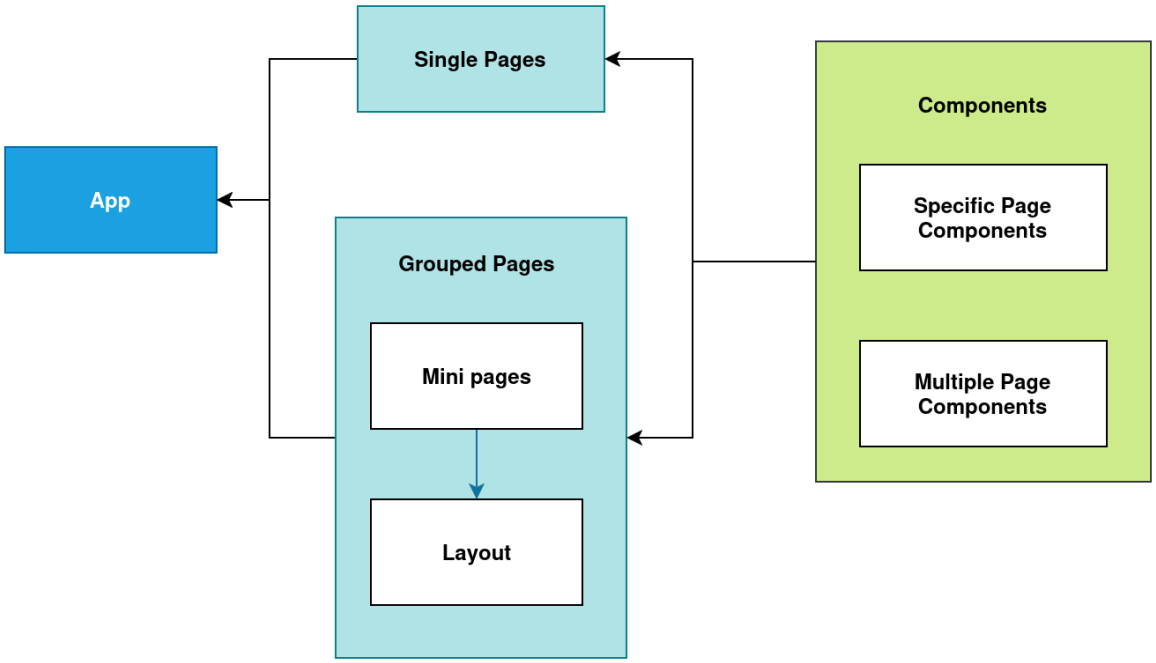

Figure 8: Architecture diagram of the Laptop Management Store frontend system

The diagram illustrates the `App` as the central structure, connecting to **Single Pages** and **Grouped Pages**. **Single Pages** are standalone, while **Grouped Pages** (e.g., Admin and Customer Pages) are organized under layouts (`Mini pages`, `Layout`). Both page types link to **Components**, split into Specific Page Components (unique to each page) and Multiple Page Components (shared across pages).

Single Pages: Single Pages are standalone, each with specific components:

- `HomePage.jsx`: Uses `ImageGallery.jsx`, `ProductSlider.jsx`, `TabProductSlider.jsx`, `PostCard.jsx`, `PostCardGridLayout.jsx` for products and ads.
- `CatalogPage.jsx`: Uses `BrandsSection.jsx`, `FilterSection.jsx` to organize products.

- `ProductPage.jsx`: Includes `ProductImage.jsx`, `ProductTabs.jsx`, `Purchase.jsx`, `SupportSection.jsx` for product details.
- `ShoppingCartPage.jsx`: Uses `ShoppingItemsTable.jsx` for cart items.
- `PlaceOrderPage.jsx`: For finalizing orders, no subcomponents.
- `CustomerLoginPage.jsx`: Customer login, no subcomponents.
- `RegisterPage.jsx`: User registration, no subcomponents.
- `SearchPage.jsx`: Product search, no subcomponents.

Grouped Pages: Grouped Pages include **Admin Page** and **Customer Page**, each with layouts and tabbed subpages:

- **Admin Page**: Under `AdministratorLayout.jsx`, includes:
  - `AdminDashboardTab.jsx`: Uses `Dashboard.jsx`.
  - `AdminOrdersTab.jsx`: Uses `OrderTable.jsx`.
  - `AdminRefundRequestsTab.jsx`: Uses `RefundTable.jsx`.
  - `AdminStockAlertsTab.jsx`: Uses `StockAlertTable.jsx`.
  - `AdminInventoryTab.jsx`: No subcomponents.
  - `AdminProductDetailTab.jsx`: No subcomponents.
- **Customer Page**: Under `CustomerLayout.jsx`, includes:
  - `CustomerOrderTab.jsx`: Uses `CustomerOrderTable.jsx`.
  - `CustomerAccountInformationTab.jsx`: No subcomponents.
  - `CustomerProductReviewsTab.jsx`: No subcomponents.

Multiple Page Components: Reusable components shared across pages:

- `WebsiteHeader.jsx`: Header for navigation.
- `WebsiteFooter.jsx`: Footer for links.
- `ProductCard.jsx`: Displays product info, used in `HomePage.jsx`, `CatalogPage.jsx`.

4.1.2 API Integration

This part details the integration of three key external services into the frontend: **FastAPI** for backend communication, **Firebase** for authorization, and **Ant Design** for UI component integration.

- **FastAPI (Backend API)**: The frontend communicates with the backend through APIs built using FastAPI. Pages like `CatalogPage.jsx`, `ProductPage.jsx`, and `ShoppingCartPage.jsx` fetch data such as product listings, product details, and cart items.
- **Firebase (Authorization)**: Firebase is integrated for user authentication, as seen in `firebase.js` within the `utils` directory. This handles login and registration flows in `CustomerLoginPage.jsx` and `RegisterPage.jsx`. The `UserContext.jsx` provides a context API to manage user state across components, ensuring secure access to pages like `CustomerOrderTab.jsx` and `AdminDashboardTab.jsx`.
- **Ant Design (UI Components)**: Ant Design is used for UI components to ensure a consistent and responsive design. Components like tables (`OrderTable.jsx`, `RefundTable.jsx`, `CustomerOrderTable.jsx`) and forms (`CustomerLoginPage.jsx`, `RegisterPage.jsx` and `PlaceOrderPage.jsx`) leverage Ant Design's library. This integration enhances the user experience across pages, such as the filter functionality in `FilterSection.jsx` on the `CatalogPage.jsx`.

4.2 Backend

The backend system for the Laptop Management Store is engineered as RESTful API designed system for scalability, performance, and maintainability. It leverages a combination of specialized technologies to handle distinct aspects of the application's functionality, from data persistence and search to real-time operations and authentication.

4.2.1 FastAPI

FastAPI serves as the backbone of our backend, providing the framework for building all HTTP API endpoints. It handles incoming requests, routing them to the appropriate logic, processing data, interacting with other services (databases, caches), and formulating HTTP responses. All CRUD (Create, Read, Update, Delete) operations for laptops, orders, reviews, posts, and newsletter subscriptions, as well as cart management and user account creation, are exposed via FastAPI routes.

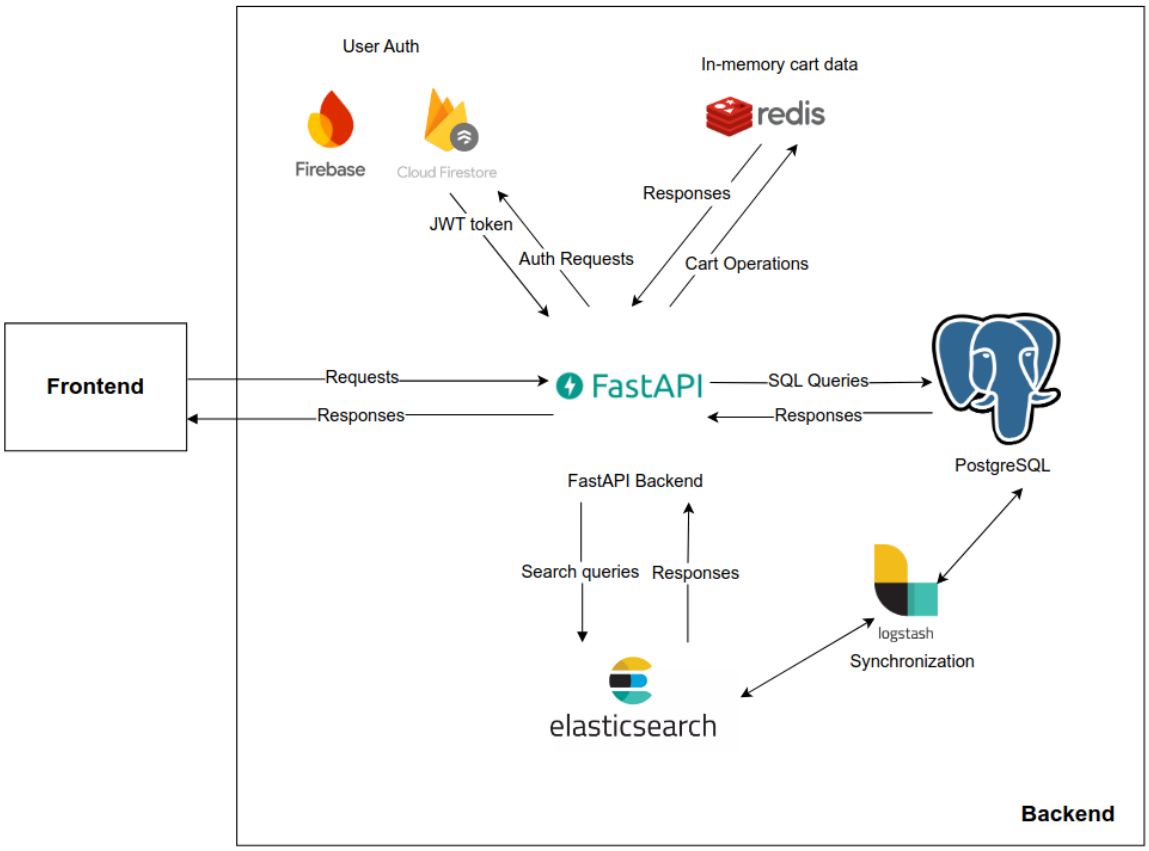

Figure 9: Architecture diagram of the Laptop Management Store backend system

- **Endpoint Design Considerations:**
  - **Resource-Oriented:** Endpoints are designed around resources (e.g., `/laptops`, `/orders`, `/cart`, `/reviews`). Standard HTTP methods (GET, POST, PUT, PATCH, DELETE) are used to represent actions on these resources.
  - **Clear Pathing:** We constructed paths logically, for example, `/orders/{order_id}` to access a specific order, or `/orders/admin/list` for administrative views.
  - **Modularity:** Routers (`APIRouter`) are used to group related endpoints into separate files (e.g., `routes/laptops.py`, `routes/orders.py`, `routes/cart.py`), which are then included in the main FastAPI application. This keeps the codebase organized and maintainable.

4.2.2 PostgreSQL

The PostgreSQL database is the cornerstone of the ITSE Laptop Shop Management System's data management, supporting a wide array of retail operations by organizing diverse information

for products, customer interactions, orders, refunds, and marketing. Its comprehensive schema, featuring multiple tables, robust relationships, and automated processes, ensures efficient storage, retrieval, and integrity for features such as product browsing, order processing, customer feedback, refund handling, and promotional activities.

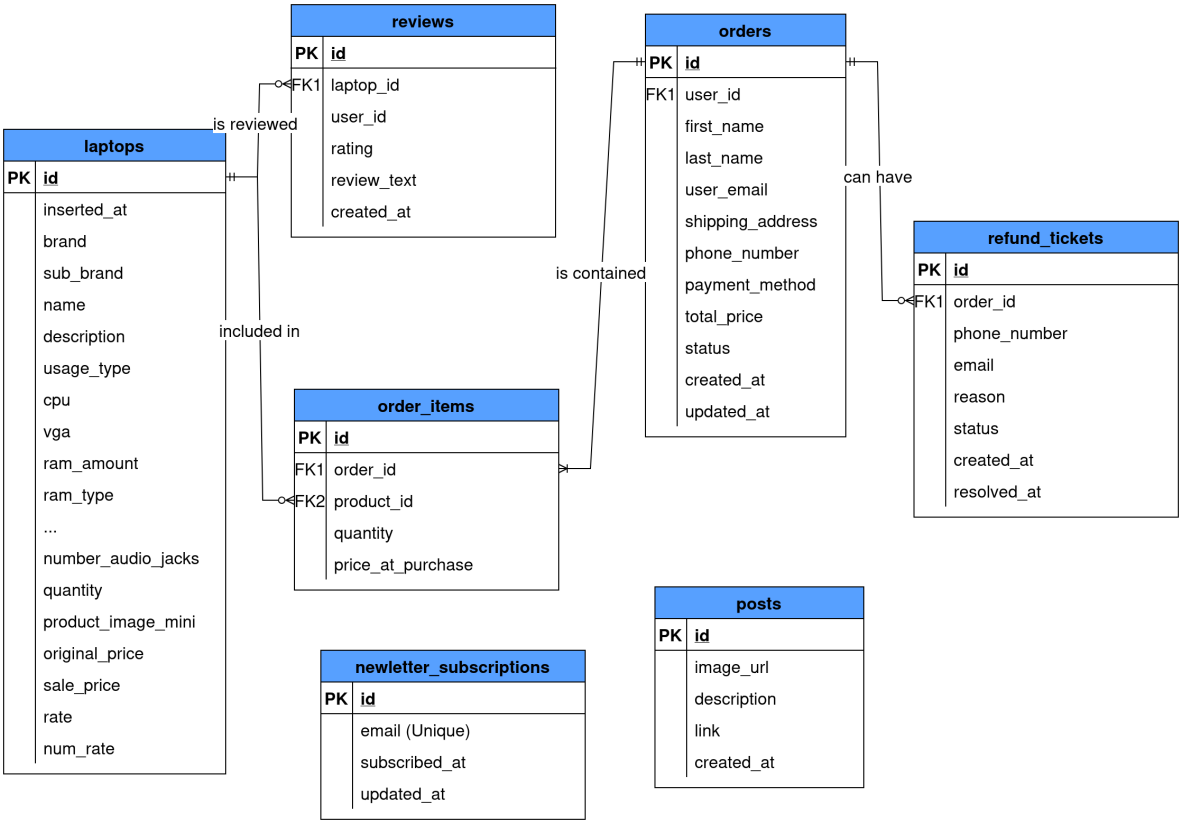

Figure 10: Entity-Relationship diagram of the Laptop Management Store database

The database includes a set of relations, each designed to address specific application needs:

- The `laptops` table stores extensive product details, including brand, model, specifications (e.g., CPU, RAM, storage type), original and sale prices, stock quantities, and customer ratings, enabling product searches and inventory updates.
- The `orders` table captures customer purchase information, such as user ID, shipping address, total price, and status (e.g., “Pending”, “Shipped”,...), facilitating order creation and tracking.
- The `order_items` table links orders to specific products, recording quantities and purchase prices to ensure accurate order breakdowns.
- The `reviews` table holds customer feedback, including ratings (1–5 stars) and comments

tied to laptops.

- The `refund_tickets` table manages refund requests, associating them with orders and tracking statuses (e.g., "Pending", "Resolved"), facilitating customer support.
- The `newsletter_subscriptions` table records email subscription, enabling targeted marketing campaigns.
- The `posts` table stores promotional content, provide customers with updates and advertisements.

This schema drives efficient operations for retail features. The `laptops` enables rapid product searches and catalog displays, while `orders` and `order_items` support order processing and dashboard analytics. The `reviews` and `refund_tickets` tables ensure effective management of feedback and support requests, with automation simplifying updates. The `newsletter_subscriptions` and `posts` tables enhance marketing efforts, making the database a versatile foundation for both operational and promotional needs, with scalability for future enhancements like new product categories or customer engagement tools.

4.2.3 ElasticSearch

Elasticsearch is employed as a secondary datastore, optimized for fast full-text search and complex filtering/aggregation of laptops, reviews, posts, and potentially orders for read-heavy operations. Data is synchronized from PostgreSQL to Elasticsearch through Logstash with the frequency of 2 minutes/update, which minimizes the amount of code and system flow, allows automatically synchronize when the PostgreSQL DB updates.

- **Advanced Full-Text Search:** Provides powerful search capabilities on product names, descriptions, and other textual fields (e.g., searching for "Ryzen 7 gaming laptop"). This is significantly faster and more flexible than using LIKE queries in PostgreSQL for large text fields. The `/laptops/search/` endpoint leverages this.
- **Complex Filtering and Faceting:** Elasticsearch excels at filtering data based on multiple criteria (e.g., brand, CPU, RAM, price range for laptops, as seen in `/laptops/filter`). It can also efficiently provide aggregations for faceted search (e.g., counts of laptops per brand matching a query).

- **Performance for Read-Heavy Queries:** For product listing pages that involve sorting and filtering across many attributes, Elasticsearch can offer better read performance than direct PostgreSQL queries, especially under load, offloading the primary database.
- **Scalability for Search:** Elasticsearch is designed to scale horizontally, making it suitable for handling a growing product catalog and increasing search traffic.

4.2.4 Firebase

- **Firebase Authentication:** Used as the primary identity provider for user registration and login. It handles user credential management, issues ID tokens (JWTs) upon successful authentication. The backend API validates these ID tokens to authenticate users.
- **Firestore:** Used to store extended user profile information (e.g., cutstomer name, detailed addresses, ... associated with a `user_id` from Firebase Auth). This data is fetch during the process of account creation.
- **Managed Authentication Service:** Offloads the complexities of building and maintaining a secure authentication system (password hashing, account recovery, OAuth integration) to a reliable third-party service. This significantly reduces development effort and security risks.
- **Scalable User Management:** Firebase Authentication can handle a large number of users without requiring direct management of user tables and credentials in our primary database.
- **Custom Claims for Authorization:** Firebase allows setting custom claims (e.g., `{"role": "admin"}`) on user tokens, which the backend API uses for role-based access control (RBAC) to protect administrative endpoints.
- **Flexible User Profile Storage (Firestore):** Firestore's NoSQL, document-based nature provides flexibility for storing diverse user profile attributes that might change over time, without requiring rigid schema migrations in PostgreSQL. This is useful for details like `first_name`, `last_name`, `address`, `phone_number`, `company`, which are then snapshotted into the PostgreSQL `orders` table at the time of purchase.

4.2.5 Redis

Redis is utilized as a high-performance, in-memory key-value store primarily for managing transient, frequently accessed data. In this project, its main role is to store user shopping carts.

Each user's cart is stored as a hash (e.g., `cart:{user_id}`) where keys are `laptop_id` and values are quantity.

- **Fast Cart Operations:** Reading and writing cart data (add item, update quantity, view cart) needs to be extremely fast for a good user experience. Redis's in-memory nature provides sub-millisecond latency for these operations.
- **Reduced Database Load:** By handling cart data in Redis, we avoid frequent read/write operations on the primary PostgreSQL database for this volatile data, preserving database resources for more critical transactional workloads like order processing and inventory updates.
- **Atomic Operations:** Redis provides atomic operations like `HINCRBY` (for incrementing item quantity in the cart), which simplifies application logic and prevents race conditions.

5 Conclusion

In conclusion, the ITSE Laptop Shop Management System successfully delivers a centralized, user-friendly platform that streamlines laptop retail operations by enhancing inventory management, simplifying sales processing, and improving customer service through a scalable React and Python-based full-stack application. Future work may include:

- Integrating additional user-friendly features to enhance the web app browsing experience.
- Refining UI/UX design to optimize the customer journey and interaction flow.
- Streamlining the frontend codebase to improve system maintainability and readability.
- Developing a comprehensive testing framework for both frontend and backend to ensure robust functionality.

Project Repository

The full source code for this project is hosted on GitHub. You can access it here:

https://github.com/auphong2707/itse-laptopshop-management-web-application